Review changelog p9-c (Phase 9 lessons 05, 06, 07)

Reviewer date: 2026-10-22. Sandbox: React/ReactDOM 19.3.1, @types/react 19.3, Vitest 3 + RTL + jsdom, TypeScript 5.9. Executed tests live in `/Users/todorbra/Projects/ReactLearning/tools/sandbox/scratch/p9-c/` (`susp.test.tsx`, `misc.test.tsx`, `act.test.tsx`, `t4` - `t7.test.tsx`, `types.tsx`); react.dev reference pages were saved to `.../p9-c/docs/`. Primary sources: react.dev reference pages (`use`, `useActionState`, `useOptimistic`, `useFormStatus`, `useTransition`, `useDeferredValue`, `Suspense`, `Activity`, `useEffectEvent`, `preload` / `preinit` / `preconnect` / `prefetchDNS`, `<form>`, `<title>`, `createContext`, `forwardRef`, `useSyncExternalStore`), React 19 / 19.1 / 19.2 / 19.3 posts and CHANGELOG saved in `scratch/p9/src/`, the installed `react-dom` development bundle, and `Object.keys(require('react'))`.

Legend: C = confirmed, X = corrected, H = hedged, U = unresolved.

05-suspense-architecture.md

Claim	How checked	Result	Change
Transition + no `key`: already-revealed content stays, `isPending` true, fallback never mounts (§3.1.2, §4.2 case 1)	Executed `susp.test.tsx`: log showed `isPending=true`, old UI visible, no `fallback mounted`	C	none
Urgent re-suspend of revealed boundary: content hidden, state preserved, layout effects cleaned up and re-run (§3.1.4)	Executed: `display: none` on content, `layout-cleanup` logged, passive `unmount-effect` NOT logged, counter state kept; react.dev Suspense Caveats	C (detail added)	Removed ⚠; added: hidden with `display: none`, passive effects not cleaned
Keyed boundary + transition: "stale content stays until new boundary resolves" (§4.2 case 2, §3.4)	Executed: `fallback mounted`, old subtree unmounted (`unmount-effect 1`), DOM showed only the fallback; react.dev "Resetting Suspense boundaries on navigation"	**X (wrong)**	Rewrote §4.2 case 2 and §3.4 bullet: `key` => fallback shown, state reset, even in a transition; clarified §5.1 note
Nested ordered reveal timings, B1 answer (Summary 600 / Timeline with Summary at 600 / Recs 900)	Executed scaled-down (60/20/90 ms) `t5.test.tsx`: Summary+Timeline appear together at ~60, Recs at ~90	C	none
React 19 pre-warms siblings after committing fallback (§3.3.3 flagged ⚠)	Executed `<A/><B/>` with A suspended: log `A render, A render, B render`; React 19 upgrade guide text	C	Removed ⚠; stated the exact behavior and that it is an optimization only
`SuspenseList` is experimental (§2.5, §3.x, Q6)	`Object.keys(require('react'))` has no `SuspenseList` / `unstable_SuspenseList`; react.dev `/reference/react/SuspenseList` 404	X (sharpened)	"unstable_SuspenseList in some builds" replaced with checked statement: not exported by stable 19.3.1, experimental channel only
SSR reveal batching (§3.6 ⚠)	React 19.2 post "Batching Suspense Boundaries for SSR"; react.dev Suspense Caveats (300 ms reveal throttle)	C	Removed ⚠, cited both
`<Activity>` introduced 19.2, stable, modes visible/hidden (§3.7)	react.dev Activity page; 19.2 post; executed `Activity` test	C	Removed ⚠; added mode names, display:none, effects destroyed/re-created, lower-priority renders
Unstable promise in `use` => never settles (§5.7, B3)	Executed with real root: 47 renders in 500 ms, fallback never replaced; react.dev `use` "fallback on every re-render"	C	Clarified
"React may log a warning about uncached promises" (B3)	Executed: no console.error in the loop; source shows the warning only fires on replay with a different promise	X (hedged)	Rewrote: warning not reliable
Symptom G: "A component suspended while responding to synchronous input"	Grepped 19.3.1 `react-dom-client.development.js`: string absent; executed urgent re-suspend: no error/warning	**X (wrong for 19)**	Rewrote Symptom G to describe the real behavior (fallback flashes silently)
`QueryErrorResetBoundary` retry hook (§3.5)	`require('@tanstack/react-query')` exports `QueryErrorResetBoundary`, `useQueryErrorResetBoundary`, `useSuspenseQuery`; docs page mentions both	C	Removed ⚠

Claim	How checked	Result	Change
`createResource` cache: rejections not cached, retry works (§5.2/5.4)	Reasoned from code; not executed	H	Removed ⚠ from "Limitations" only (they are limitations, not unverified claims)
Links	curl: all 200; CLS page has no "Optimize CLS" heading	X	Section changed to "How to improve CLS"; removed " (unverified)" markers

Unresolved in 05: SSR/streaming/hydration table (§3.6) and "fallback duration via performance marks" (§5.8) were not executed (no browser); AsyncBoundary with `react-error-boundary` not installed/executed.

06-concurrent-rendering-in-practice.md

Claim	How checked	Result	Change
`useTransition` sync sequence	Executed: `App render n=0 pending=true` -> `Slow render n=0` (isPending render) -> `App n=1 pending=false` + `Slow n=1` ; StrictMode doubles every line	C	Added Strict Mode evidence to §3.1.9
Updates after `await` in async transition are not part of the transition (§3.1.3, §3.5, A5, Q6)	react.dev useTransition ("currently require wrapping..."); executed: un-wrapped `setN` rendered with `pending=true` as its own render, then `pending=false` ; re-wrapped landed in one commit	C	Removed all ⚠, added evidence
`useDeferredValue` render counts, with/without `initialValue`	Executed (non-strict / strict): no initialValue -> 1 render on mount, update `dv=old` then `dv=new` ; with `"INIT"` or `""` -> mount renders `dv=INIT` then `dv=a`	C	Added the counts to §3.1.4
Deferred value that suspends keeps old content, no fallback	Executed (`resA` stayed, then `resB`); react.dev caveat	C	Added
`useDeferredValue` inside a Transition returns new value immediately	react.dev caveat (not executed)	C (doc)	Added
Multiple async transitions batched (§3.5 ⚠)	react.dev useTransition/useActionState text; executed two separate `useTransition` hooks: fast one did not commit until slow one finished	C	Rewrote with evidence
19.3 "Transitions render independently, no longer entangled"	CHANGELOG 19.3; executed sync transitions: A suspended on slow promise, B's update committed (`b=1`) while A pending	C	Added nuance: independence holds for sync transitions that suspend, not for parallel async Actions (still batched in 19.3.1)
Async transition throwing -> Error Boundary	Executed: boundary showed `caught:nope`	C	none
B2: ref write in render under Strict Mode	Executed: `0 / 2` after mount, `1 / 4` after click	C	Added numbers
B3: getSnapshot returning new object	Executed: console.error "The result of getSnapshot should be cached to avoid an infinite loop", then throws "Maximum update depth exceeded"	X (detail)	Replaced vague "warning" with exact behavior
uSES forces blocking re-render when store mutates during transition (§3.1.6, §3.4)	react.dev useSyncExternalStore caveats	C	none
Performance tracks in Chrome, 19.2 (§3.8)	19.2 post "Performance Tracks"; react.dev reference page 200	C	Removed ⚠; added Scheduler-track detail and link
`react-compiler` ESLint rules (F1)	react.dev nav lists lints `purity` , `refs` , `immutability` , `set-state-in-render` under `eslint-plugin-react-hooks` ; 19.2 post says plugin v6	X (updated)	Reworded
Links	curl: all 200; discussion #69 title is "What is tearing?"	C	Removed "(unverified)" markers

Unresolved in 06: the §4.1 / B1 timeline (commit counts and times for the 5,000-row search) and the "expiration/starvation" claim (§2.6) were not executed (jsdom has no real scheduler timing); the Profiler "caused by transition" flagging varies by DevTools version (kept as hedge).

07-modern-react-apis.md

Claim	How checked	Result	Change
Version notice: `useEffectEvent` / `Activity` / `cacheSignal` /Perf Tracks "⚠ later minors"	19.2 post; 19.3 post and CHANGELOG (ViewTransition, Fragment Refs, browser() stable in 19.3)	X (outdated)	Rewrote notice with stable versions
`useActionState` signature `[state, formAction, isPending]`, `(prev, payload)`, permalink, queue	react.dev; `tsc --noEmit` on `types.tsx` (FormData and string payloads)	C	Added: dispatch must run in an Action; throw cancels the queue and hits the boundary; `useFormState` alias still exported
Queued actions (§3.2, 5.1, B1 answer)	Executed: with 3 queued actions, renders were `count=0 pending=true` x3 then ONE `count=3 pending=false`; intermediate counts never rendered; input reset only after the last	**X**	Corrected B1 ("input resets after each", per-action commits), §3.2 note, §5.1 prediction (also fixed the contradiction with `disabled={isPending}`)
`useFormStatus` returns `{pending,data,method,action}`, parent-form rule	Executed: child saw `pending=true method=get data=v action=function`, then `pending=false method=null`; react.dev	C	none
`useOptimistic` revert on success/failure, reducer form, setter must be in Action	Executed (§3.3, §4.1, §5.2 patterns, LikeButton C1): overlay shown during Action, single commit to real state; failure reverts and shows error; outside-transition warning text captured	C	Removed ⚠, added evidence and exact warning text
Form reset after Action; `defaultValue` from returned state keeps input (§4.2, §5.3)	Executed with echo/no-echo variants: reset re-applies the new `defaultValue` (`ABC`) or empties; reset also happens after a failure-state return	C	Removed ⚠, added evidence
`requestFormReset`	`react-dom` 19.3.1 exports it and @types/react-dom types it; react.dev page 404	H	Stated: exported, no reference page; suggested `form.reset()` per react.dev example
`use(promise)` with stable vs unstable promise	Executed (see 05)	C	Replaced "React warns/loops" with checked behavior
`use` **in try/catch "does NOT catch rejection" (§2.4, §5.11)**	Executed: the `catch` DID catch the rejection and rendered its fallback, with DEV warning "`use` was called from inside a try/catch block..."; react.dev says not allowed	**X**	Rewrote to "unsupported; React warns"
Conditional `use(context)` after early return	Executed: works, no warnings	C	none
19.3 DEV warning for conditional `use()` that unblocks	CHANGELOG only; could not trigger in a simple test	U	Mentioned with "exact trigger unverified"

Claim	How checked	Result	Change
Ref as prop, ref callback cleanup lifecycle (§3.5 table)	Executed: mount `ref(el)`; identity change -> cleanup then `ref(el)`; unmount -> cleanup only; non-cleanup callback gets `ref(null)`	C	Added executed lifecycle
TS error on implicit-return ref callback	`tsc` with `@ts-expect-error` (error confirmed), message `... not assignable to ... VoidOrUndefinedOnly`; upgrade guide codemod `no-implicit-ref-callback-return`	C	Removed ⚠ and "verify codemod"; named codemod
`<Context value>`; `.Provider` deprecation	react.dev createContext ("legacy way... before React 19"), React 19 post ("will deprecate")	X (reworded)	Replaced "planned for deprecation ⚠"
`forwardRef` deprecation	react.dev forwardRef page	C	Cited
Stylesheet `precedence` + Suspense	react.dev Suspense page ("blocks the boundary until the stylesheet loads, up to a timeout")	C	Removed ⚠
`preload` / `preinit` / `preconnect` / `prefetchDNS` callable in render	react.dev reference pages (render-time OK; SSR only effective in render or async context from render); all six functions exist in `react-dom` 19.3.1 and typecheck	C	Replaced ⚠
"If multiple components render `<title>`, the last rendered wins"	react.dev `<title>`	**X**	Replaced: all are placed in head, behavior undefined
`useEffectEvent` rules; stable in 19.2	react.dev; executed: no reconnect on theme change, latest value seen; throws "can't be called during rendering"; typecheck	C	Removed ⚠; added allowed call sites (useEffect/useLayoutEffect/useInsertionEffect)
`<Activity>` semantics (§3.8, §5.13)	Executed: hidden mount renders but runs no effects; hide cleans up effects, keeps state, `display: none`; show re-runs effects	C	Removed ⚠, replaced "behavior to verify" with verified behavior
Q3 "why `use` can be conditional" explanation	Read `trackUsedThenable` in react-dom dev bundle (thenable state indexed per render attempt, not committed hook state)	X (reworded)	Rewrote answer
Root options `onCaughtError` / `onUncaughtError` / `onRecoverableError`	react.dev createRoot page mentions all three	C	none
Links	curl: all 200	C	Removed unverified notes

Unresolved in 07: server-side metadata/stylesheet/preload emission (§3.6 SSR column) and no-JS progressive enhancement were not executed; `permalink` behavior requires a framework with Server Functions (not executable here); Server Component snippets (§5.6) not executed.

Totals

Confirmed: 38. Corrected: 14 (keyed boundary + transition; nonexistent React 18 warning in Symptom G; queued-actions commit/reset behavior in 07 B1/§5.1; `use` try/catch claim; `<title>` last-wins; outdated experimental/Canary notices; `SuspenseList` status; B3 uncached-promise warning; getSnapshot error detail; Q3 explanation; `.Provider` deprecation wording; ESLint compiler rules; CLS link section; async-transition batching nuance for 19.3). Unresolved/hedged: 8 (conditional-`use` warning trigger, `requestFormReset` docs, SSR/streaming behaviors, performance marks, 06 §4.1 timeline, starvation/expiration, Profiler flagging, `permalink`).